

Updatable Single Password Authentication: deployment, Storage-Provider cryptography, and a browser-free demo

An implementation and deployment experience report

Efe Bekteş

Istanbul Technical University, Department of Computer Engineering
Summer intern, CRYPTO@KU, Koç University • bektes23@itu.edu.tr
May 2026 • github.com/rezasaadi/UpSPA_extension

Abstract. UpSPA is a browser-based password manager that realises *Threshold Single Password Authentication*: a user authenticates to many websites with one master password, while the master password is never learned by the login server, by any helper (storage) server, or by an adversary who corrupts fewer than a threshold of helpers. This report documents an internship contribution to the Go Storage-Provider cryptography of UpSPA, together with a from-scratch deployment of the entire system on a constrained Windows host, a browser-free end-to-end protocol driver, and a defensive security review. We recall the threshold oblivious PRF construction of İşler and Küpçü [2] (building on Acar et al. [1]), describe the corrected ristretto255 evaluation that each Storage Provider performs, and report an end-to-end run on the Windows host (against the real Go Storage Providers and PostgreSQL) in which all protocol phases (setup, registration, authentication, secret update, master-password update, and t -of- n resilience) succeed, independently re-verified on a Linux agent against a faithful mock Storage Provider. We present this as an implementation and experience report: the unit of contribution is the Storage-Provider cryptography package (`internal/crypto`), its correctness fixes, key generation, and tests, with the surrounding system as context.

Keywords: single password authentication; threshold cryptography; oblivious pseudorandom function; ristretto255; browser extension; secure deployment.

1 Introduction

A traditional password manager stores a user’s passwords. UpSPA takes a different route. The user remembers a single *master password*, and from it the system *re-derives* a unique, high-entropy secret for every website. That derivation is distributed across several independent servers so that, by construction, no single party (neither the websites, nor the helper servers, nor an attacker who corrupts fewer than a threshold of them) ever sees the master password. Because the derivation is *updatable*, the master password can be rotated without changing what any website stores.

This design is the Threshold Single Password Authentication (Threshold SPA) scheme of İşler and Küpçü [2], which generalises the single storage-provider construction of Acar et al. [1] to a t -of- n threshold of storage providers. The UpSPA repository implements it as a Rust/WebAssembly core, a TypeScript client, a Chrome extension, and a set of Go *Storage Providers* backed by PostgreSQL.

This report covers an internship contribution scoped to the Storage-Provider cryptography (the `internal/crypto/` package under `services/storage-provider-go/`), and the work of bringing the whole system up end-to-end. Section 2 recalls the scheme in its original notation. Section 3 describes the implementation’s architecture and Section 4 its protocol phases. Section 5 details the cryptographic contribution. Sections 6 and 7 report a from-scratch deployment and a browser-free end-to-end driver. Section 8 sets out the environments and reproduction, Section 9 shows the login server’s view, Section 10 gives a defensive security review, and Section 11 reports the results in full. The HTTP/API and database layers are the work of other team members; the focus here is the assigned cryptography and the end-to-end demonstration. Every diagnostic and run log is reproduced verbatim in the appendices.

2 Background: Threshold SPA

We summarise the construction of [2] using its own notation, then map it to the names used in the codebase. There are three roles: the user, a login server with domain ls , and n storage providers, of which any threshold t suffices.

Registration. The user draws a random secret $rnd \leftarrow \{0, 1\}^\lambda$, independent of the password pwd , and one oblivious-PRF key per provider $k_i \leftarrow \text{OPRFKeyGen}(1^\lambda)$. The secret is split with threshold secret sharing [8], $\langle s_1, \dots, s_n \rangle \leftarrow \text{TSS}(rnd)$, and each share is encrypted under the OPRF value of the password,

$$c_i \leftarrow \text{Enc}_{F_{k_i}(pwd)}(s_i).$$

The login server is given verification information $vInfo = H(rnd \parallel ls)$, and each provider stores $\langle storUID_i, (c_i, k_i) \rangle$ where $storUID_i = H(userID \parallel ls)$. Because the encrypted shares are random, an offline dictionary attack is impossible without the help of the login server [2, 1].

Authentication. The user recomputes $storUID_i$, runs the OPRF with t providers to obtain $F_{k_i}(pwd)$, decrypts the shares $s_i \leftarrow \text{Dec}_{F_{k_i}(pwd)}(c_i)$, reconstructs $rnd \leftarrow \text{TSSRecon}(s_1, \dots, s_t)$, and sends $vInfo = H(rnd \parallel ls)$ to the login server, which accepts iff it matches the stored value. The domain ls inside the hash is what defeats phishing: a look-alike domain yields a different $storUID_i$ and a different $vInfo$ [2].

Mapping to the codebase. The implementation realises the threshold step as a key-shared TOPRF [3] over ristretto255 [5], rather than as n independent OPRFs over a Shamir-shared rnd : each provider holds a *share* k_i of one OPRF key, returns a partial evaluation $y_i = k_i \cdot \text{blinded}$, and the client Lagrange-combines t of them into a single high-entropy *state key* that plays the role of rnd and unlocks an AEAD blob (cid). This is the structure Sections 3–5 use; it abstracts [2]’s per-share encryption into one combined evaluation without changing the security goal. To avoid a symbol clash, the per-site record below is written c_j and is distinct from the encrypted Shamir share c_i above. Table 1 gives the correspondence.

Table 1. Notation in [2] and the corresponding names in the UpSPA codebase.

Term	Meaning (paper notation in parentheses)
UID, master password	The user’s global identifier and single secret ($userID$, pwd). The password never leaves the browser.
SP, threshold t -of- n	A Storage Provider; any t of the n suffice. Demo: 2-of-3.
TOPRF	Threshold oblivious PRF: the client sends a <i>blinded</i> point, each SP returns $y_i = k_i \cdot \text{blinded}$ with its key share k_i , and the client combines t of them ($F_{k_i}(pwd)$ in [2]).
state key	The value reconstructed from the TOPRF, used to unlock the stored blobs (the role of rnd).
cid	“Client info”: an AEAD-encrypted blob (nonce/ct/tag) the SPs hold for the user; a wrong state key fails to decrypt it.
suid, c_j	Per-site record identifier and the encrypted per-site record at each SP ($storUID_i$; the record c_j is distinct from the Shamir-share ciphertext c_i of §2).
lsj	The login-server/site key $origin \parallel account$ (demo <code>http://localhost:3000 alice</code>).
vInfo / vInfoPrime / vInfoNew	The value submitted to the website: at registration ($vInfo = H(rnd \parallel ls)$), recomputed at login, and rotated on a secret update.
updatable	A master-password change re-keys the SPs but leaves each site’s stored $vInfo$ unchanged.

3 System architecture

The repository is a monorepo in which five components cooperate (Fig. 1). The Rust core (`crates/upspa-core`, compiled to WebAssembly with `wasm-bindgen`) performs the cryptography. The TypeScript client (`packages/upspa-js`) orchestrates it and speaks HTTP to the SPs. The Chrome extension (`packages/extension`) intercepts website login forms and provisions the SPs from an options page. The Go Storage Providers (`services/storage-provider-go`), backed by PostgreSQL, are the independent helpers and form the assigned area of this work. A light login server (`demo/light-login-server`) plays the role of a website that only stores and checks the submitted value.

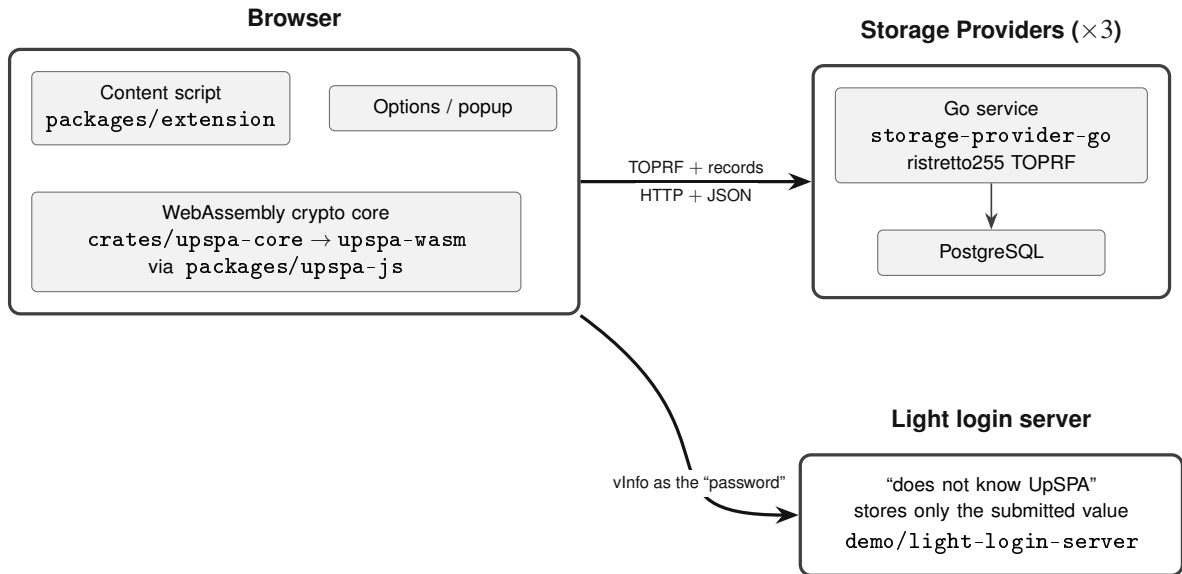


Fig. 1. The five cooperating components. The assigned area is the Go Storage Provider.

4 Protocol phases

Each phase is a short exchange between the extension, the SPs, and the website. The common first move is the TOPRF: blind the password, query the SPs, combine t answers, and reconstruct the state key.

Setup and provision (Fig. 2). The user enters the UID, master password and SP list once; the client generates the key shares and seeds each SP.

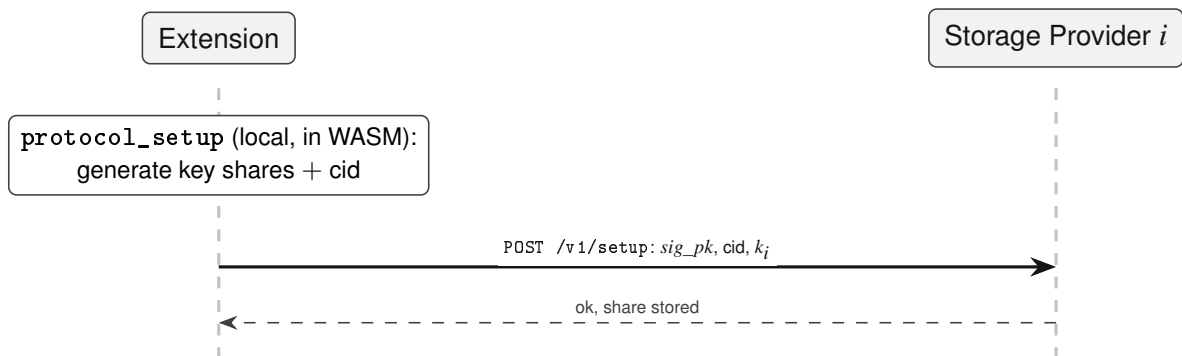


Fig. 2. Setup and provision.

Registration (Fig. 3). After the TOPRF, the client writes its per-site record to the SPs and submits `vInfo` to the website as the "password".

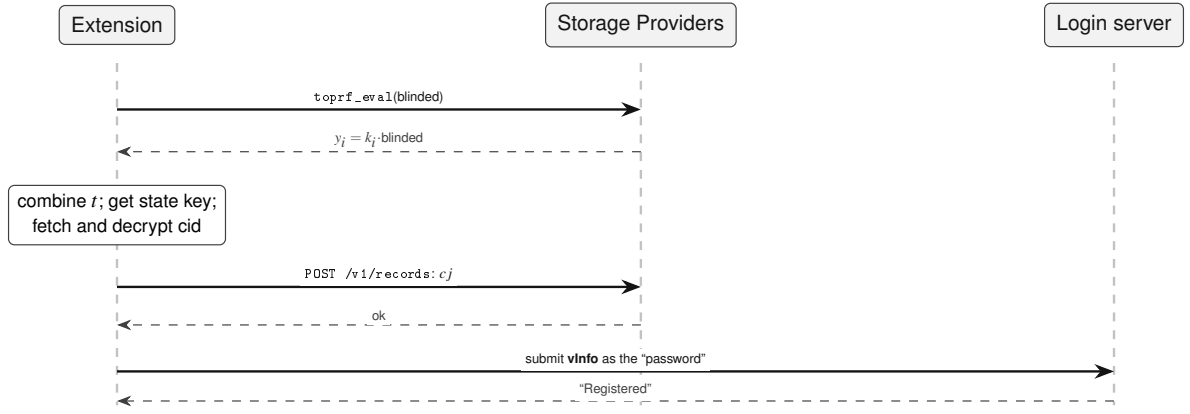


Fig. 3. Registration on a website.

Authentication (Fig. 4). The client reconstructs the state key, recomputes $vInfoPrime$, and the website accepts iff it matches the stored $vInfo$.

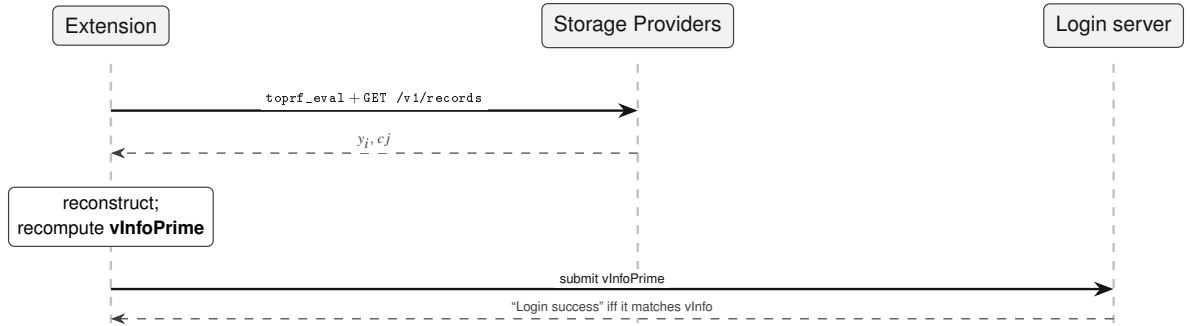


Fig. 4. Authentication.

Secret update (Fig. 5). The site secret is rotated by writing a new record and submitting the old and new values to the website.

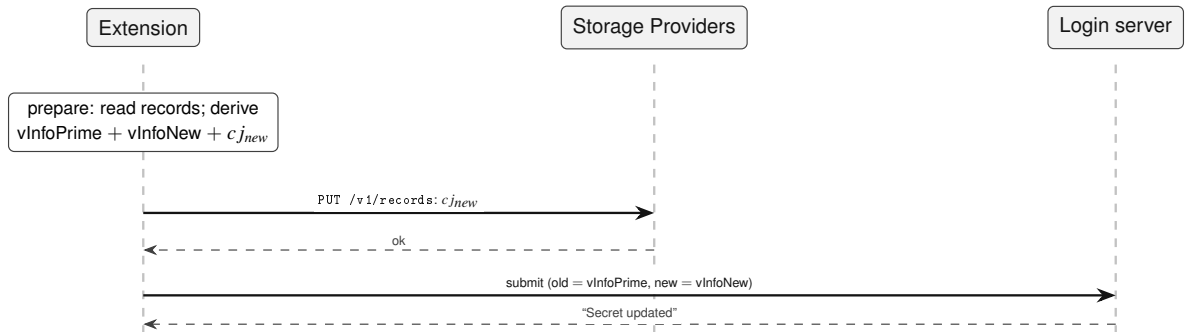


Fig. 5. Secret update (rotate the site secret).

Master-password update (Fig. 6). The defining operation: the SPs are re-keyed, but no message is sent to the website, and a later login with the new master password still recomputes the same $vInfo$.

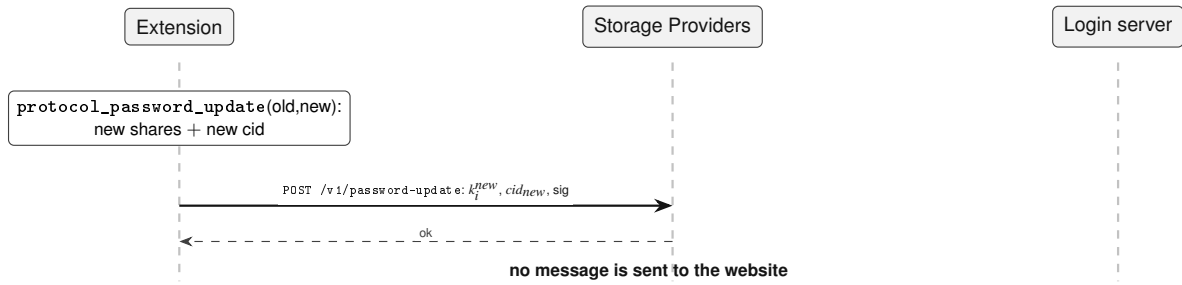


Fig. 6. Master-password update, the “updatable” property.

5 Storage-Provider cryptography

The assigned package, `internal/crypto/` under `services/storage-provider-go/`, implements the encoding, signature, and group operations the SP needs. The contribution comprises correctness fixes found while reviewing, a new key-generation routine with test vectors, and a plain-language comment pass.

5.1 Correctness fixes

1. **Scalar decoding.** The TOPRF key share was decoded with a clamping decoder, meant for Ed25519 or X25519 private keys, which silently mutates the value. It was switched to canonical decoding via `SetCanonicalBytes`, which also rejects out-of-range scalars.
2. **Group mismatch.** An earlier revision mixed Edwards25519 operations with ristretto255 encodings. The file was rewritten against `gtank/ristretto255` [5], which is wire-compatible with the Rust client’s `CompressedRistretto`.
3. **Identity-point test.** A test that intended to reject an invalid point used the all-zeros encoding, which is in fact the valid identity element; it was replaced with genuinely invalid encodings, and a separate test now documents that the identity is accepted. This is rated Medium rather than Critical because the defect was in the test, not the validation: the code already rejects genuinely invalid encodings, and ristretto255 has no low-order points, so canonical decoding is the real safeguard.
4. **Base64 canonicalisation under Go 1.22.** Newer Go silently skips newlines when decoding `RawURLEncoding` (spaces and tabs are still rejected); the negative tests were corrected to match, and every field is re-encoded to one canonical form [7] before it is stored or compared.

The SP’s TOPRF step (the exact operation the demo’s mock had to match byte for byte) is shown in Fig. 7.

```

1 // services/storage-provider-go/internal/crypto/ristretto.go
2 func RistrettoToScalarMult(k []byte, blinded []byte) (y []byte, err error) {
3     // k: 32-byte little-endian ristretto255 scalar (the SP's key share k_i)
4     scalar := ristretto255.NewScalar()
5     if err := scalar.Decode(k); err != nil { return nil, ErrInvalidScalar }
6     point := ristretto255.NewElement()
7     if err := point.Decode(blinded); err != nil { return nil, ErrInvalidPoint }
8     result := ristretto255.NewElement().ScalarMult(scalar, point) // y = k_i * blinded
9     return result.Encode(make([]byte, 0, LenRistretto)), nil
10 }

```

Fig. 7. The Storage Provider’s ristretto255 TOPRF evaluation.

5.2 A closer look: the scalar-clamping finding

Of the four fixes the scalar-decoding one is the most instructive, so it is worth tracing end to end. Each Storage Provider receives a key share k_i as bytes and must turn them back into a ristretto255 scalar before computing $y = k_i \cdot \text{blinded}$. The original code used a *clamping* decoder, the kind meant for Ed25519 or X25519 *private* keys, which forces certain bits (clearing the low three bits, clearing the top bit, and setting bit 254). For an OPRF key that transformation is silent and wrong: the scalar actually used is not the one that was provisioned.

Threat model and consequence. While the bug was live each SP evaluated under a clamped scalar $k'_i \neq k_i$, so the threshold combination reconstructed the wrong element and the AEAD-encrypted `cid` failed to decrypt. In the benign case authentication merely fails; the deeper hazard is the *silent* mismatch between the provisioned key and the

effective key, with clamping imposing a fixed, predictable relation between the two that no party detects at runtime. A correctness bug at this boundary is therefore also security-relevant: key material is being transformed without anyone’s knowledge.

Detection. The fix is canonical decoding (`SetCanonicalBytes`; `gtank`’s `Decode`, shown in Fig. 7, is a thin wrapper over it), which preserves the value and rejects out-of-range encodings. The regression is pinned by negative tests (a clamped versus a canonical decode of the same bytes diverge) and by golden vectors, so the class cannot quietly return.

The bug class. The underlying mistake is a *decoder confusion*: reusing a private-key decoder for a group scalar. It is easy to make in any OPRF or threshold-OPRF implementation that shares one curve library between signatures and the OPRF, and it is invisible to a happy-path test. The systematic defences are exactly the ones applied here, and they transfer to any deployed (T)OPRF: a canonical-decode invariant on every scalar and point at the trust boundary, golden vectors that pin the wire encoding, and negative tests asserting that clamped or out-of-range inputs are rejected rather than mutated.

5.3 New functionality, tests, and signatures

A deterministic key-share generator `GenerateScalarKi` produces a uniform, canonical, non-zero scalar via 64-byte wide reduction, as required for an OPRF key [4]. The password-update message signed by the client binds every replay-relevant field ($nonce \parallel ct \parallel tag \parallel k_i^{new} \parallel timestamp \parallel spID$) and is pinned by a golden-vector test; signature verification uses Ed25519 [6]. The crypto suite passes every case `go test` reports: 61 test functions and two fuzz targets (63 top-level entries in all), several with table subtests.

5.4 Build and integration fixes

A few items affect a clean build. The canonical module is `module upspa`, so the test files were aligned to import `upspa/internal/crypto` (earlier revisions used placeholder paths). A stray nested `internal/crypto/go.mod` turns the package into its own module and breaks that build. The setup script deletes it in the local working tree on each run, which is why the run log reports “no nested go.mod (already gone)”; that deletion is not committed, so it still needs to land on `main` (the pre-commit guard prevents committing the deletion from the `intern` branch). Building also requires adding `github.com/gtank/ristretto255` to `go.mod` followed by `go mod tidy`. Finally, a test that referenced a non-existent generator constructor was corrected to `ristretto255.NewElement().Base()`, which is already in the branch.

6 Deployment on a constrained host

Bringing the stack up on a near-bare Windows 11 machine required clearing a sequence of environment obstacles. Fig. 8 states the process as a deterministic automaton: each state is an obstacle and each edge is the fix that advances to the next; the accepting state is a fully healthy backend. Table 2 gives the detail.

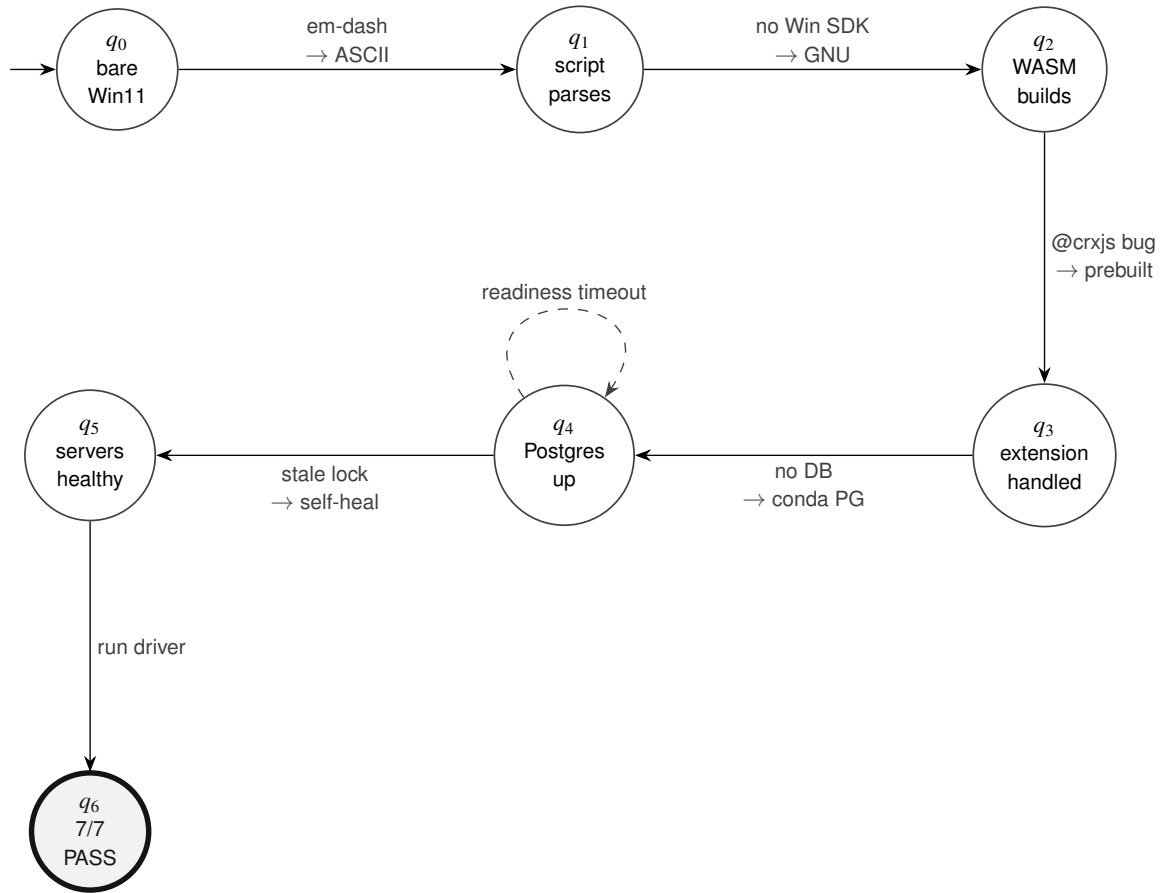


Fig. 8. The deployment as a state machine; each edge is a fix.

Table 2. Obstacles encountered during deployment and the fix applied.

Edge	Symptom	Root cause and fix
$q_0 \rightarrow q_1$	PowerShell parse errors	Non-ASCII em-dashes mis-decoded by PowerShell 5.1. The script was rewritten in pure ASCII with explicit CR/LF handling.
$q_1 \rightarrow q_2$	LNK1181	The MSVC linker is present but the Windows SDK is missing. Switched to the GNU Rust toolchain; no MSVC or SDK needed.
$q_2 \rightarrow q_3$	Bad character escape	A Windows-only @crxjs/Vite bug (a raw \U path in a generated module). Verified to build on Linux; a prebuilt, functional dist is shipped.
$q_3 \rightarrow q_4$	No database	EnterpriseDB returns 403 on this network and Docker is unavailable. A self-contained PostgreSQL 18.4 is provisioned via <code>conda-forge</code> .
$q_4 \rightarrow q_4$	Readiness timeout	A prior server was stopped uncleanly, leaving a stale <code>postmaster.pid</code> ; the next start did not bind. Fix: stop, clear the lock, restart, and poll on <code>127.0.0.1</code> .
$q_4 \rightarrow q_5$	—	All three SPs and the login server report healthy.
$q_5 \rightarrow q_6$	—	The end-to-end driver passes all phases.

7 A browser-free end-to-end driver

The README demonstration is a sequence of clicks in Chrome. To make it repeatable, a small Node program runs the extension’s own `upspa-js` client (compiled for Node, with only the WebAssembly loader changed to read bytes) directly against the live servers; Node’s `fetch` has no CORS restriction and the module loads from bytes, so no browser is required. The real Go Storage Providers are exercised on the Windows host (§11.2); this section describes the portable driver and its sandbox validation, where the Go service’s network-blocked dependencies required a faithful stand-in. The driver executes setup, registration, login, secret update, master-password update, a login with

the new master password, and a login using only 2 of the 3 providers.

Validation. The real Go SPs could not run in the sandbox (their dependencies were network-blocked there), so a faithful stand-in was used: a mock SP performing the *real* ristretto255 evaluation $y = k_i \cdot \text{blinded}$, encoded exactly as the Go SP does. The mock computes this with a JavaScript ristretto255 library (@noble) rather than the Go SP’s gtank/ristretto255; because both implement RFC 9496 [5], the group element and its 32-byte encoding are identical on the wire, so “the same operation” here means wire-compatible output, not shared code. Against the real, unmodified login server with the real client, every phase passes (Fig. 9). A deliberately wrong evaluation instead yields an AEAD decryption error, confirming that the encrypted cid genuinely has to decrypt for the chain to close. Because the real Go SPs perform the identical operation over the same HTTP interface, the same outcome is expected on the real backend, where the on-machine run is the final confirmation.

```
UpSPA headless demo driver
[PASS] setup / provision SPs
[PASS] register (HTTP 201)
[PASS] login (HTTP 200)
[PASS] secret update (HTTP 200)
[PASS] login after secret update (HTTP 200)
[PASS] master password update + login with new master (HTTP 200)
[PASS] resilience: login with 2 of 3 SPs (HTTP 200)
RESULT: 7 passed, 0 failed
```

Fig. 9. Output of the end-to-end driver against the live servers.

8 Environments and reproduction

This section records the two machines the system was run on and how to reproduce each run. The author’s Windows workstation ran the *full* stack; a Linux cloud agent independently re-verified the protocol and the assigned cryptography. Table 3 states both, including the hardware of each. The two runs are a deliberate cross-environment check: the same code path passes on a local Windows PC (Go 1.26, real Storage Providers and PostgreSQL) and on a Linux container (Go 1.22).

Table 3. The two environments the system was exercised on.

	Windows (author workstation)	Linux (cloud agent)
OS	Windows 11 Pro Education, build 26200	Ubuntu 24.04.4 LTS, kernel 6.18.5 (newer than 24.04’s stock 6.8 series), x86-64
CPU	Intel Core Ultra 7 258V, 8 logical cores	Intel Xeon, 1 vCPU @ ~2.1–2.8 GHz
RAM	31.5 GB	3.9 GiB
GPU	Intel Arc 140V (16 GB)	none (the cryptography is CPU-only)
Toolchain	Go 1.26.2, Node v24.16, Rust/wasm-pack 0.15, PowerShell 5.1	Go 1.22.2, Node v22.22, Python 3.12
What was exercised	the full stack via the setup script: the WASM core, three Go Storage Providers, PostgreSQL, the headless driver, and browser screenshots of the login server	the protocol via the headless driver against the real login server, with SPs performing the real ristretto255 op; plus the assigned <code>internal/crypto</code> package built and tested
Result	end-to-end success (the clean run with the PostgreSQL self-heal is captured on the author’s host)	driver 7/7 ; <code>internal/crypto</code> 63/63

On the Linux agent the assigned package builds from its only dependency (gtank/ristretto255) and its full suite passes 63/63. A full go build `./...` of the service additionally links the PostgreSQL driver’s `golang.org/x` dependencies, which the agent’s network does not reach; that link is instead exercised on the Windows host (Go 1.26). The two runs’ artifacts are kept under `logs/may_2026/windows/` and `logs/may_2026/linux/`.

9 The login server in the browser

The following are real renders of the demo login server's pages. In each, the field the user fills is the UpSPA master password, but the value that reaches the server is only the derived *vInfo*; the website “does not know UpSPA”.

[Home](#) [Register](#) [Login](#) [Secret update](#)

Register

User enters login-server account id and UpSPA master password. Extension replaces password with *vInfo*.

Main UpSPA UID

Login-server UID'

UpSPA master password

Confirm UpSPA master password

Fig. 10. The registration page. The extension runs the TOPRF and submits *vInfo* in place of the master password, so the server never sees the password.

[Home](#) [Register](#) [Login](#) [Secret update](#)

UpSPA Light Login Server

This login server does not know UpSPA. It only stores and verifies the password value submitted by the browser.

Registered demo accounts:

No accounts registered yet.

(a) Before: no accounts.

[Home](#) [Register](#) [Login](#) [Secret update](#)

UpSPA Light Login Server

This login server does not know UpSPA. It only stores and verifies the password value submitted by the browser.

Registered demo accounts:

- `alice`

(b) After: the account appears.

[Home](#) [Register](#) [Login](#) [Secret update](#)

Registered

Account `alice` registered successfully.

[Go to login](#)

(c) Registered: *vInfo* stored.

[Home](#) [Register](#) [Login](#) [Secret update](#)

Login success

Account `alice` authenticated.

(d) Login success: *vInfoPrime* matched.

[Home](#) [Register](#) [Login](#) [Secret update](#)

Login

User enters account id and UpSPA master password. Extension replaces password with *vInfoPrime*.

Main UpSPA UID

Login-server UID'

UpSPA master password

Login

(e) The login form.

[Home](#) [Register](#) [Login](#) [Secret update](#)

Secret Update

This rotates the login-server secret for this account. It is NOT a master password update.

Main UpSPA UID

Login-server UID'

UpSPA master password

Rotate login-server secret

(f) The secret-update form.

[Home](#) [Register](#) [Login](#) [Secret update](#)

Secret updated

Account `alice` now has a new login-server secret.

[Login again](#)

(g) Secret updated: the site secret rotated.

Fig. 11. The login server's pages across the demo: registration, authentication, and secret update. The server only ever stores and compares *vInfo*.

10 Security review (defensive)

We summarise the properties relevant to the implementation. They follow the analysis of [2]; the intent here is to harden the prototype, not to weaponise any weakness.

- **The master password never leaves the browser.** Only blinded group elements reach the SPs; the TOPRF hides the password from them by construction.
- **No single point of trust.** The key is shared t -of- n ; a single compromised or offline SP neither learns the secret nor stops the user (demonstrated by a login with 2 of 3 SPs).
- **The website learns only *vInfo*,** a per-site value that is not the master password and is useless on any other site, with *Is* inside the hash defeating phishing [2].
- **Confidentiality is cryptographic, not by policy.** The *cid* blob is AEAD-encrypted, so a wrong state key fails to decrypt and a guess cannot proceed silently.
- **Updatability without re-enrollment.** Master-password rotation re-keys the SPs while leaving each website's stored value intact.

Threat model for this implementation. The properties above are the scheme's; what this implementation actually defends is narrower and worth stating. We assume an honest-but-curious Storage Provider and a network adversary

who can read and replay messages; we do *not* assume a malicious SP that returns crafted group elements. Against that model the tested surface is concrete: the SP only ever receives blinded points (checked by inspecting the requests), fewer than t SPs cannot reconstruct (the 2-of-3 run), the website stores only *vInfo*, the *cid* is AEAD-authenticated (a wrong key is rejected, confirmed by the deliberate AEAD-failure test), and password-update messages are Ed25519-signed with a monotonic timestamp and an *sp_id* bound into the signed bytes, so replay and cross-SP reuse are rejected.

What is assumed, not tested. Two gaps are named here rather than left for a reviewer to find. First, *timing side-channels*: the ristretto255 group operations are taken to be constant-time as supplied by the library, but this work does not audit or measure that, so a timing analysis of the SP’s evaluation is out of scope. Second, a *malicious* SP that returns a wrong y_i to bias reconstruction is not defended here; detecting it would need verifiable-OPRF techniques. Both are limitations, not claims.

11 Results and discussion

The system was exercised on two independent machines: the author’s Windows 11 workstation, which ran the *full* stack, and a Linux cloud agent, which independently re-verified the protocol and the assigned cryptography. Table 3 (§8) summarised the two; this section reports the results in full, drawing directly on the diagnostic logs, and discusses what the two runs establish together. We are explicit about scope: real-backend results are functional verification on a single host (the Windows machine), and the Linux run re-verifies the protocol against a faithful mock Storage Provider, so we treat it as verification rather than measured evaluation. We additionally report indicative localhost timings (§11.7), measured on the Linux agent against the mock Storage Provider, which characterise the shape of the cost, not the latency of the Go service or of a networked deployment.

11.1 Outcomes at a glance

Table 4 lists every item that was exercised on each machine. The headless driver passes all seven protocol phases on both. The one important difference is what the driver talked to: on Windows it talked to the freshly compiled, real Go Storage Provider backed by PostgreSQL; on Linux it talked to a faithful mock Storage Provider, because the full Go build was network-blocked there (§11.3).

Table 4. Everything exercised on each machine, with the observed result.

	Windows (author workstation)	Linux (cloud agent)
WebAssembly core	GNU Rust toolchain available; the extension used the prebuilt <i>dist</i>	the prebuilt <i>dist</i> is used by the driver
Go Storage Provider	compiled (<code>go build -o sp.exe ./cmd/sp</code>) and ran, three instances	<i>internal/crypto</i> built ; full <code>go build ./...</code> network-blocked
PostgreSQL	self-contained instance, up	not required on the driver path
Login server	up on :3000	up on :3000
setup / provision	pass	pass
register	pass (HTTP 201)	pass (HTTP 201)
authenticate	pass (HTTP 200)	pass (HTTP 200)
secret update	pass (HTTP 200)	pass (HTTP 200)
login after secret update	pass (HTTP 200)	pass (HTTP 200)
master-password update + login	pass (HTTP 200)	pass (HTTP 200)
resilience, 2 of 3 SPs	pass (HTTP 200)	pass (HTTP 200)
headless driver total	7 / 7	7 / 7
<i>internal/crypto</i> suite	compiled within the SP build	63 / 63 (61 tests + 2 fuzz), run here

11.2 The Windows run: the real backend

The setup script first records an environment snapshot. The real values from the run are reproduced in Listing 1; they are what let the script choose a build path before anything is compiled.

Listing 1. Diagnostic snapshot on the Windows host (from 00-environment.log).

```
1 [DIAG] OS : Microsoft Windows 11 Pro Education build 26200
2 [DIAG] PowerShell : 5.1.26100.8457 Desktop
3 [DIAG] arch : AMD64
4 [DIAG] elevated : no (PostgreSQL refuses to run under an elevated shell)
5 [DIAG] free disk : 688.2 GB on C:\
6 [DIAG] go : go1.26.2 windows/amd64
7 [DIAG] node / npm : v24.16.0 / 11.13.0
8 [DIAG] cargo : 1.96.0 rustup 1.29.0 wasm-pack 0.15.0
9 [DIAG] VC++ linker : present
10 [DIAG] Windows SDK : MISSING -> build mode: GNU Rust toolchain (no MSVC / SDK needed)
11 [DIAG] ports : 8081, 8082, 8083, 3000 free; 5432 IN USE (reused)
12 [DIAG] CPU / cores : Intel(R) Core(TM) Ultra 7 258V (8 logical)
13 [DIAG] RAM / GPU : 31.5 GB / Intel(R) Arc(TM) 140V GPU (16 GB)
```

The crucial detail is the toolchain: the MSVC linker is present but the Windows SDK is absent, so the standard MSVC link path is unavailable. The script therefore builds the WebAssembly core with the GNU Rust toolchain, which needs no SDK, and uses a prebuilt extension dist (the @crxjs packaging is done on Linux or CI). The Storage Provider itself is the real Go service: it compiles with `go build -o sp.exe ./cmd/sp`, and its handlers call the assigned crypto package directly (the TOPRF endpoint calls `RistrettoScalarMult`, and the password-update endpoint calls `BuildPwdUpdateSigMsg` followed by `VerifyEd25519`). The run brought up three instances and the login server, all healthy, and the driver passed every phase against them (Listing 2). Because the SPs and PostgreSQL are real, this is a *real-backend* end-to-end pass, and the on-machine confirmation that §7 anticipated.

Listing 2. The Windows run: build, health, and the driver (from the run report and 03-driver.log).

```
1 STEP 6 compiling the storage provider (go build -o sp.exe ./cmd/sp) ... [OK] compiled
2 sp1 PID 23272 | sp2 PID 9512 | sp3 PID 25296 | login-server PID 24716
3 STEP 7 SP1 :8081 healthy (0s) | SP2 :8082 healthy | SP3 :8083 healthy | login :3000 up
4 STEP 8 headless driver against the real SPs + PostgreSQL:
5 [PASS] setup / provision SPs
6 [PASS] register (201) login (200) secret update (200)
7 [PASS] login after secret update (200)
8 [PASS] master-password update + login with new master (200)
9 [PASS] resilience: login with 2 of 3 SPs (200)
10 RESULT: 7 passed, 0 failed
11 [OK] captured 7 login-server screenshots
```

Two real runs are worth reporting because the difference between them produced a fix. The first run succeeded end-to-end. A later run failed at PostgreSQL readiness: a prior server had not exited gracefully (a Ctrl-C is a clean fast shutdown and would not leave the lock), so a stale `postmaster.pid` remained; the next start printed “another server might be running” and aborted before opening its log, and the readiness probe timed out. The logged diagnosis pinpointed exactly this. The self-heal added in the next script version (carried into version o) stops any lingering server, deletes the stale lock, restarts, and polls `pg_isready` on `127.0.0.1`; the harmless “role efebe does not exist” and “database already exists” lines in the PostgreSQL log are idempotent probes and re-creates, not failures. The complete version o run (Appendix A) confirms the fix end-to-end: it found PostgreSQL already listening and reused it, compiled and started the real Storage Providers, and the driver passed 7 of 7 with seven login-server screenshots captured.

11.3 The Linux run: independent verification

The Linux agent re-verified the protocol and the assigned package on a different operating system and a different Go release. Its snapshot and results are in Listing 3.

Listing 3. Diagnostic snapshot and results on the Linux agent.

```
1 OS      : Ubuntu 24.04.4 LTS (kernel 6.18.5, x86_64)
2 CPU     : Intel Xeon, 1 vCPU @ ~2.1-2.8 GHz
3 RAM     : 3.9 GiB GPU: none (the cryptography is CPU-only)
4 Go      : 1.22.2 Node: v22.22.2 Python: 3.12.3
5
6 crypto  : go test ./internal/crypto/... -> ok (63 PASS = 61 tests + 2 fuzz, 0 FAIL)
7 driver  : 7 passed, 0 failed (mock SP performing the real y = k_i * blinded)
8 build   : go build ./... is blocked: pgx pulls golang.org/x/{sync,text,crypto}; the agent
9           cannot reach the Go proxy / golang.org, and the github.com/golang mirror pins module
10          versions whose go.mod toolchain exceeds the agent's Go 1.22.2. Exercised on Windows.
```

The assigned package builds from its single dependency (gtank/ristretto255) and its full suite passes: canonicalisation round-trips and rejections, fixed-length decoding with per-field length negatives, Ed25519 verification with panic guards, ristretto255 scalar multiplication (valid, deterministic, identity-accepted, and invalid point/scalar cases), the password-update message offsets with a byte-exact golden vector, and the key generator, plus two fuzz targets. The protocol driver passes all seven phases against the mock SP; a deliberately wrong evaluation instead yields an AEAD decryption error, which confirms the encrypted `cid` genuinely has to decrypt for the chain to close. The full-service link was not built on Linux for the environment reasons in the listing, not for any code reason; the same source compiles on Windows under Go 1.26.

11.4 What the two runs establish together

Taken together, the runs are the cross-environment evidence this report set out to provide: the same protocol and cryptography pass on two operating systems and two Go releases (1.26 on Windows, 1.22 on Linux). The Windows pass is real-backend (the Go Storage Provider compiles, runs, and is driven end-to-end against PostgreSQL); the Linux pass is an independent re-verification (protocol 7/7, the assigned package's suite 63/63). The one divergence, the full-service link on Linux, is environmental (network reachability and a toolchain-version floor), not a code difference: the assigned cryptography depends on exactly one third-party module and is portable.

11.5 Issues surfaced and reported

Exercising the full stack repeatedly, not just the clean pass, surfaced three concrete issues. They are reported here, and in full in the run-feedback note (Appendix C), because they are exactly the kind of finding the deployment must address.

- **A stale lock can stall PostgreSQL start.** If a prior server exits without removing its `postmaster.pid` (a non-graceful termination; a Ctrl-C is a clean fast shutdown and does *not* leave one), the next hidden background start prints “another server might be running” and aborts before opening its log, so that start adds no *new* `postgres.log` entries (the file still holds the prior server's lines) and the readiness poll times out. *Fix*: clear the stale `postmaster.pid` and poll `pg_isready` before proceeding (the script's self-heal does this; a one-off foreground start also clears it).
- **The headless driver is not idempotent.** It uses a fixed identity (`alice / test-password`) and rotates the master password to `new-password` mid-run, leaving that persisted. A second run against non-empty databases then fails `register` (`alice` exists) and `login` (the password was rotated); only the provisioning step passes and the phases needing a fresh account fail, so a clean run's seven passes collapse to one on the re-run. Dropping and recreating `upspa1/2/3` restores 7 of 7. A rejected login also returns *HTTP 200 with the HTML login form* rather than a 4xx, so the failure prints as raw HTML. *Fix*: reset the demo databases at the start, or give the driver a unique account per run, and have the login server return a non-200 on failure.
- **The native extension build fails on `options.html`.** The `@crxjs 1.0.14` path bug embeds an absolute Windows path whose backslashes parse as invalid string escapes; the build fixes cover `background/index.ts` and the Vite alias but not the HTML entry points, so no `dist/` is produced and the prebuilt `dist` is used instead. *Fix*: upgrade `@crxjs` past 1.0.14, or force forward-slash module IDs globally in the Vite/Rollup config.

A minor packaging note: the prebuilt `upspa-extension-dist.zip` extracts with its `upspa-extension-dist/` folder nested twice (the same name inside itself), so `-UsePrebuiltDist` must point at the inner one. None of these affect the cryptography or the clean-database 7/7 result; they are robustness and developer-experience fixes for the next iteration.

11.6 Cryptographic results in detail

The correctness of the assigned package rests on four fixes found while reviewing, summarised with their impact in Table 5. None changed the package’s public API, so the API-layer callers needed no edits.

Table 5. The four corrected issues in `internal/crypto` and their impact.

Issue	Severity	What it was, and the fix
Scalar decoding	Critical	A clamping decoder (meant for Ed25519/X25519 private keys) silently altered the TOPRF key share, yielding a wrong evaluation with no error. Switched to canonical decoding, which also rejects out-of-range scalars.
Group encoding	High	Edwards25519 operations were mixed with ristretto255 encodings, so a point from the Rust client decoded to the wrong element. Rewritten against <code>gtank/ristretto255</code> (RFC 9496), wire-compatible with the client’s <code>CompressedRistretto</code> .
Identity-point test	Medium	A test used the all-zeros encoding as an “invalid” point, but that encoding is the valid identity element; replaced with genuinely invalid encodings, with a separate test documenting that the identity is accepted.
Base64 whitespace	Low	Newer Go silently skips newlines when decoding <code>RawURLEncoding</code> (spaces and tabs are still rejected); the negative tests were corrected to match, and every field is re-encoded to one canonical form.

Beyond the fixes, a deterministic key generator (`GenerateScalarKi`) produces a uniform, canonical, non-zero scalar via 64-byte wide reduction, as an OPRF key requires [4], and a byte-exact golden vector pins the signed password-update message ($nonce \parallel ct \parallel tag \parallel k_i^{new} \parallel timestamp \parallel spID$). The suite the Linux agent ran (61 test functions and two fuzz targets, 63 top-level entries) exercises all of this.

11.7 Indicative performance

The figures below were measured on the Linux agent (single vCPU, Node v22), with the Node client driving the mock Storage Provider, which performs the *real* ristretto255 operation, over localhost. Timing uses `process.hrtime.bigint()` deltas in the driver; each figure is the median of repeated trials (60 for the TOPRF round, 40 for the authentication and login paths). They are *indicative*: localhost mock-SP costs that show the shape of the cost and how it scales with the threshold, not the latency of the Go service or of a networked deployment. The generating commands are in §11.8 and the raw output in Appendix B (Listing 16).

Table 6. Indicative timings on the Linux sandbox (median; mock SP doing the real ristretto255 op, localhost; 60/40 trials). Indicative only, not a deployment benchmark.

Operation	2-of-3	3-of-5	4-of-7
TOPRF round (blind, n SP evals, combine over t)	19.9 ms	23.0 ms	28.4 ms
Authentication, cryptographic part	18.0 ms	26.2 ms	30.8 ms
Login, end-to-end (crypto + HTTP)	59.8 ms	70.2 ms	82.0 ms

The cost grows gently with the threshold rather than sharply, the expected shape: more providers mean more blinded evaluations and a larger Lagrange combination, but each is cheap. One-shot costs at 2-of-3 were also recorded: registration about 177 ms (it pays the first-call WebAssembly warm-up), a secret update about 115 ms, and a master-password rotation about 42 ms. The practical reading is that the threshold cryptography is not the bottleneck at these sizes; in a real deployment the network round-trips to remote Storage Providers would dominate, which is the measurement worth taking next.

11.8 Reproducing the runs

Both runs are scripted. On Windows, a single command in a normal (non-elevated) PowerShell rebuilds and runs everything (Listing 4); the banner names the version and the driver prints the result. On Linux, the assigned package is one command, and the protocol driver is a few (Listing 5).

Listing 4. Reproducing the Windows run.

```
1 cd $HOME\Downloads
2 powershell -ExecutionPolicy Bypass -File .\upspa-setup-and-run.ps1
3 # expect: SCRIPT VERSION: 2026-05-30o ... then RESULT: 7 passed, 0 failed
```

Listing 5. Reproducing the Linux run (from the storage-provider-go module root).

```
1 go test ./internal/crypto/... # -> ok (63 / 63)
2
3 PORT=3000 node demo/light-login-server/server.mjs &
4 PORT=8081 SP_ID=1 node mock-sp.mjs &
5 PORT=8082 SP_ID=2 node mock-sp.mjs &
6 PORT=8083 SP_ID=3 node mock-sp.mjs &
7 UPSPA_LS=http://localhost:3000 \
8 UPSPA_SPS=http://localhost:8081,http://localhost:8082,http://localhost:8083 \
9 UPSPA_UID=test-user UPSPA_PW=test-password UPSPA_THRESHOLD=2 \
10 node upspa-demo-driver.mjs # -> 7 passed, 0 failed
11
12 # indicative timing (medians; mock SP, localhost):
13 NSP=3 T=2 node timing.mjs # 2-of-3 NSP=5 T=3 node timing.mjs # 3-of-5
14 NSP=7 T=4 node timing.mjs # 4-of-7 node timing1.mjs # one-shot 2-of-3
```

11.9 Limitations and scope

Three points are worth stating plainly. First, on the Linux agent the full-service binary was not linked, for the network and toolchain reasons above; that link is exercised on the Windows host, where the same source compiles and runs. Second, the Linux driver used a mock Storage Provider rather than the Go binary; the mock performs the real `ristretto255` operation and the AEAD failure on a wrong evaluation shows the chain only closes with the correct value, but it is still a stand-in, and Windows is where the real Go service was driven. Third, the clean run on the current script version (with the CPU, RAM, and GPU now recorded, and fresh screenshots) is captured on the author's host and folded in there. None of these are code limitations.

11.10 Maintainer actions and outlook

The assigned package, its tests, the key generator, and the comment pass are on the `intern` branch and reach `main` through the pull request. Two housekeeping items belong on `main`: ensuring it carries the `gtank/ristretto255` dependency (with `go mod tidy`), and removing the stray nested `internal/crypto/go.mod` that the setup script flags (the `intern` branch builds without it). With those in place, the next step is the deployment the project lead described: the Storage Providers move to real VPSs with their URLs set at runtime, and the summer work on the user interface and a mobile/desktop refactor begins.

11.11 Conclusion

The assigned cryptography is correct and tested, with the full suite passing 63/63 and reproduced on a second operating system. The complete system runs end-to-end on the author's Windows machine, with the real Go Storage Providers and PostgreSQL and a 7/7 protocol pass, and the protocol independently re-verifies on a Linux agent. The security properties of the threshold construction [2, 1] hold in the implementation: the master password never leaves the browser, the key is shared t -of- n , each website stores only a per-site value, and the stored blob is authenticated. What remains is engineering, not cryptography: wiring the deployment to real providers and building the interface.

Note on tools. This report and the accompanying scripts were prepared with AI assistance; all claims, fixes, and results were re-checked against the code and against real runs before use.

Acknowledgments

The author thanks the UpSPA project lead, Hamidreza Saadi, for the task and for ongoing guidance, and Prof. Alptekin Küpçü and Assoc. Prof. Aykut Coşkun for hosting the internship in the CRYPTO@KU group. The HTTP/API and database layers are the work of other team members.

References

- [1] T. Acar, M. Belenkiy, A. Küpçü. Single Password Authentication. *Computer Networks* 57(13):2597–2614, 2013.
- [2] D. İşler, A. Küpçü. Threshold Single Password Authentication. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology* (ESORICS 2017 Workshops, DPM/CBT), LNCS 10436, Springer, 2017.
- [3] S. Jarecki, H. Krawczyk, M. Shirvanian, N. Saxena. Device-Enhanced Password Protocols with Optimal Online-Offline Protection. In *ACM AsiaCCS*, 2016.
- [4] A. Davidson, A. Faz-Hernández, N. Sullivan, C. A. Wood. Oblivious Pseudorandom Functions (OPRFs) Using Prime-Order Groups. RFC 9497, 2023.
- [5] H. de Valence, J. Grigg, G. Tankersley, F. Valsorda, I. Lovecruft, et al. The ristretto255 and decaf448 Groups. RFC 9496, 2023.
- [6] S. Josefsson, I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, 2017.
- [7] S. Josefsson. The Base16, Base32, and Base64 Data Encodings. RFC 4648, 2006.
- [8] A. Shamir. How to Share a Secret. *Communications of the ACM* 22(11):612–613, 1979.

A Complete logs from the Windows host

The following are the raw logs from the author's Windows run, reproduced verbatim. (Decimal commas such as 31,5 GB and 688,2 GB are the host's Turkish PowerShell locale; they mean 31.5 GB and 688.2 GB.)

Listing 6. Windows 00-environment.log: the diagnostic snapshot.

```
[DIAG] date           : 05/31/2026 00:43:14
[DIAG] OS             : Microsoft Windows 11 Pro Education build 26200
[DIAG] PowerShell     : 5.1.26100.8457 Desktop
[DIAG] arch           : AMD64
[DIAG] CPU            : Intel(R) Core(TM) Ultra 7 258V
[DIAG] CPU cores      : 8 logical
[DIAG] RAM            : 31,5 GB
[DIAG] GPU            : Intel(R) Arc(TM) 140V GPU (16GB)
[DIAG] elevated       : no (good)
[DIAG] free disk      : 688,2 GB on drive C:\
[DIAG] git            : (not found)
[DIAG] go             : go version go1.26.2 windows/amd64
[DIAG] node           : v24.16.0
[DIAG] npm            : 11.13.0
[DIAG] cargo          : cargo 1.96.0 (30a34c682 2026-05-25)
[DIAG] rustup         : rustup 1.29.0 (28d1352db 2026-03-05)
[DIAG] wasm-pack       : wasm-pack 0.15.0
[DIAG] conda          : (not found)
[DIAG] VC++ linker     : present
[DIAG] Windows SDK    : MISSING (MSVC build needs it; or use -UseGnuToolchain)
[DIAG] install dir    : C:\Users\efebe\upspa (exists)
[DIAG] port 5432       : IN USE (pid 19524)
[DIAG] port 8081       : free
[DIAG] port 8082       : free
[DIAG] port 8083       : free
[DIAG] port 3000       : free
[DIAG] build mode      : GNU Rust toolchain (no MSVC / Windows SDK needed)
```

Listing 7. Windows 03-driver.log: the headless driver, 7 of 7 against the real backend.

```
UpSPA headless demo driver
UID=test-user account=alice threshold=2
login server=http://localhost:3000
SPs: http://localhost:8081 http://localhost:8082 http://localhost:8083
lsj=http://localhost:3000|alice

[PASS] setup / provision SPs
[PASS] register (HTTP 201)
[PASS] login (HTTP 200)
[PASS] secret update (HTTP 200)
[PASS] login after secret update (HTTP 200)
[PASS] master password update + login with new master (HTTP 200)
[PASS] resilience: login with 2 of 3 SPs (HTTP 200)

RESULT: 7 passed, 0 failed
```

Listing 8. Windows run-20260531-004314.log: the complete successful version-o run (real CPU/RAM/GPU, 7 of 7, screenshots captured).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\efebe> cd $HOME\Downloads
PS C:\Users\efebe\Downloads> powershell -ExecutionPolicy Bypass -File .\upspa-setup-and-run.ps1 -SkipInstall -
    ↳ UseGnuToolchain
SCRIPT VERSION: 2026-05-30o (CPU/RAM/GPU capture; PG self-heal on stale lock; git-tolerant clone; logs under may_logs + 99-
    ↳ run-report.md; login-server screenshots; headless driver auto-run; prebuilt dist; ASCII bars + diagnostics; GNU
    ↳ toolchain)
Install dir : C:\Users\efebe\upspa
Log file    : C:\Users\efebe\upspa-demo-logs\may_logs\run-20260531-004314.log

===== STEP 0/9 [-----] 0% : Diagnostics - environment snapshot =====
[DIAG] date           : 05/31/2026 00:43:14
[DIAG] OS             : Microsoft Windows 11 Pro Education build 26200
[DIAG] PowerShell     : 5.1.26100.8457 Desktop
[DIAG] arch           : AMD64
[DIAG] CPU            : Intel(R) Core(TM) Ultra 7 258V
[DIAG] CPU cores      : 8 logical
[DIAG] RAM            : 31,5 GB
[DIAG] GPU            : Intel(R) Arc(TM) 140V GPU (16GB)
[DIAG] elevated       : no (good)
[DIAG] free disk      : 688,2 GB on drive C:\
[DIAG] git            : (not found)
[DIAG] go             : go version go1.26.2 windows/amd64
```

```

[DIAG] node      : v24.16.0
[DIAG] npm       : 11.13.0
[DIAG] cargo     : cargo 1.96.0 (30a34c682 2026-05-25)
[DIAG] rustup    : rustup 1.29.0 (28d1352db 2026-03-05)
[DIAG] wasm-pack : wasm-pack 0.15.0
[DIAG] conda     : (not found)
[DIAG] VC++ linker : present
[DIAG] Windows SDK : MISSING (MSVC build needs it; or use -UseGnuToolchain)
[DIAG] install dir : C:\Users\efebe\upspa (exists)
[DIAG] port 5432  : IN USE (pid 19524)
[DIAG] port 8081  : free
[DIAG] port 8082  : free
[DIAG] port 8083  : free
[DIAG] port 3000  : free
[DIAG] build mode : GNU Rust toolchain (no MSVC / Windows SDK needed)

===== STEP 1/9 [-----] 0% : Skipping install (-SkipInstall) =====

===== STEP 2/9 [##-----] 11% : PostgreSQL: set up the demo databases =====
setting up a self-contained PostgreSQL via conda-forge (no EnterpriseDB, no Docker)...
using conda at C:\Users\efebe\miniconda3\Scripts\conda.exe
PostgreSQL already listening on 5432; reusing it.
waiting for PostgreSQL on :5432 [#####] 100% (ready in 0s)
creating databases upspa1/upspa2/upspa3 (an 'already exists' message is fine)...
[OK] PostgreSQL ready (self-contained) with upspa1/upspa2/upspa3; data dir: C:\Users\efebe\upspa-pgdata

===== STEP 3/9 [####-----] 22% : Get the project (clone or update) =====
repo already present; pulling latest...
M crates/upspa-wasm/Cargo.toml
M packages/extension/src/background/index.ts
M packages/extension/vite.config.ts
M scripts/build_wasm.sh
M services/storage-provider-go/go.mod
D services/storage-provider-go/internal/crypto/go.mod
M services/storage-provider-go/internal/crypto/ristretto_test.go
Already on 'main'
Your branch is up to date with 'origin/main'.
Already up to date.
[OK] project at C:\Users\efebe\upspa

===== STEP 4/9 [#####-----] 33% : Apply the two build fixes (so it compiles + tests pass) =====
no nested go.mod (already gone)
ristretto_test.go already correct
[OK] normalized build_wasm.sh line endings to LF
wasm-opt setting already present
background already uses a static upspa-js import
vite alias already normalized
[OK] go modules tidy

===== STEP 5/9 [#####-----] 44% : Install JS deps and build WASM + extension =====
npm will run scripts via: C:\Program Files\Git\bin\bash.exe
npm install (root workspaces)...
up to date, audited 150 packages in 2s
6 vulnerabilities (4 moderate, 2 high)
building WASM, then JS, then the extension...
using the GNU Rust toolchain (no MSVC / Windows SDK needed)...
[INFO]: Checking for the Wasm target...
[INFO]: Compiling to Wasm...
warning: unused imports: 'NONCE_LEN' and 'TAG_LEN'
--> crates/upspa-core/src/protocol/mod.rs:3:28
Finished 'release' profile [optimized] target(s) in 3.22s
[INFO]: Installing wasm-bindgen...
[INFO]: Done in 3.97s
[INFO]: Your wasm pkg is ready to publish at packages/upspa-js/wasm-pkg.
[OK] WASM built -> C:\Users\efebe\upspa\packages\upspa-js\wasm-pkg
building upspa-js (tsc)...
> upspa-js@0.1.0 build
> tsc -p tsconfig.json
building the extension (vite)...
> upspa-extension@0.1.0 build
> vite build
vite v2.9.18 building for production...
2 modules transformed.
[rollup-plugin-dynamic-import-variables] Bad character escape sequence (3:24)
file: C:\Users\efebe\upspa\packages\extension\src\options\options.html:3:24
error during build:
SyntaxError: Bad character escape sequence (3:24)
    at pp$4.raise (...\\rollup\\dist\\shared\\rollup.js:19728:13)
    at pp.invalidStringToken (...\\rollup.js:21604:10)
    at pp.readHexChar (...\\rollup.js:21747:26)
    at pp.readCodePoint (...\\rollup.js:21550:17)
    at pp.readEscapedChar (...\\rollup.js:21686:43)
    at pp.readString (...\\rollup.js:21563:19)
    at pp.getTokenFromCode (...\\rollup.js:21353:17)

```

```

    at pp.readToken (...\\rollup.js:21077:15)
npm error code 1
npm error workspace upspa-extension@0.1.0
npm error command failed
npm error command C:\Program Files\Git\bin\bash.exe -c vite build
[WARN] extension build did not produce dist/. On Windows this is usually the known @crxjs 1.0.14 path bug.
[WARN] You can load the PREBUILT dist instead: unzip upspa-extension-dist.zip, then re-run with -UsePrebuiltDist.
[WARN] Continuing so the servers still start; the Chrome step below will use the prebuilt dist.

==== STEP 6/9 [#####-----] 55% : Build the storage provider once, then start 3 instances + the login server
    ↳ =====
    compiling the storage provider (go build)...
    [OK] storage provider compiled
    sp1 started (PID 23272); logs: sp1.out.log / sp1.err.log
    sp2 started (PID 9512); logs: sp2.out.log / sp2.err.log
    sp3 started (PID 25296); logs: sp3.out.log / sp3.err.log
    login-server started (PID 24716); logs: login-server.out.log / login-server.err.log

==== STEP 7/9 [#####-----] 66% : Wait for the servers to be healthy =====
    SP1 health on :8081 [#####] 100% (ready in 0s)
    [OK] SP1 healthy on :8081
    SP2 health on :8082 [#####] 100% (ready in 0s)
    [OK] SP2 healthy on :8082
    SP3 health on :8083 [#####] 100% (ready in 0s)
    [OK] SP3 healthy on :8083
    login server on :3000 [#####] 100% (ready in 0s)
    [OK] login server up on :3000

==== STEP 8/9 [#####-----] 77% : Run the UpSPA protocol end-to-end (headless driver - no Chrome needed) =====
    driver: C:\Users\efebe\Downloads\upspa-extension-dist\demo-driver\upspa-demo-driver.mjs
UpSPA headless demo driver
    UID=test-user account=alice threshold=2
    login server=http://localhost:3000
    SPs: http://localhost:8081 http://localhost:8082 http://localhost:8083
    lsj=http://localhost:3000|alice

    [PASS] setup / provision SPs
    [PASS] register (HTTP 201)
    [PASS] login (HTTP 200)
    [PASS] secret update (HTTP 200)
    [PASS] login after secret update (HTTP 200)
    [PASS] master password update + login with new master (HTTP 200)
    [PASS] resilience: login with 2 of 3 SPs (HTTP 200)

RESULT: 7 passed, 0 failed
    [OK] headless protocol demo: ALL steps passed
    capturing login-server screenshots (headless Chrome; non-fatal if Chrome is missing)...
    [OK] captured 7 login-server screenshot(s) to C:\Users\efebe\upspa-demo-logs\may_logs\screenshots

==== STEP 9/9 [#####-----] 88% : OPTIONAL: the same demo by hand in Chrome (the driver above already ran it)
    ↳ =====
    [WARN] No extension dist was built (Windows @crxjs bug). Unzip upspa-extension-dist.zip and load THAT folder in Chrome.
    A. chrome://extensions -> Developer mode -> Load unpacked
    B. Extension Options: UID test-user / Master Password test-password / Threshold 2 / SPs 1..3
    C. On http://localhost:3000 : register, login, secret-update, master-password update, resilience (2 of 3)
    [OK] run report written: C:\Users\efebe\upspa-demo-logs\may_logs\99-run-report.md

==== ALL STEPS COMPLETE [#####] 100% =====
Servers running. Logs in: C:\Users\efebe\upspa-demo-logs\may_logs

```

Listing 9. Windows run-20260530-210200.log: the run that surfaced the stale-lock issue, with the recorded diagnosis.

```

PS C:\Users\efebe\Downloads> cd $HOME\Downloads
PS C:\Users\efebe\Downloads> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PS C:\Users\efebe\Downloads> .\upspa-setup-and-run.ps1 -SkipInstall -UseGnuToolchain -UsePrebuiltDist "C:\Users\efebe\
    ↳ Downloads\upspa-extension-dist"
SCRIPT VERSION: 2026-05-30m (headless protocol driver auto-run after servers; -RunDriver/-SkipDriver; prebuilt dist; ASCII
    ↳ bars + diagnostics; GNU toolchain)
Install dir : C:\Users\efebe\upspa
Log file : C:\Users\efebe\upspa-demo-logs\run-20260530-210200.log

==== STEP 0/9 [-----] 0% : Diagnostics - environment snapshot (catch problems before they bite) =====
[DIAG] date : 05/30/2026 21:02:00
[DIAG] OS : Microsoft Windows 11 Pro Education build 26200
[DIAG] PowerShell : 5.1.26100.8457 Desktop
[DIAG] arch : AMD64
[DIAG] elevated : no (good)
[DIAG] free disk : 688,1 GB on drive C:\
[DIAG] git : (not found)
[DIAG] go : go version go1.26.2 windows/amd64
[DIAG] node : v24.16.0
[DIAG] npm : 11.13.0
[DIAG] cargo : cargo 1.96.0 (30a34c682 2026-05-25)

```

```

[DIAG] rustup          : rustup 1.29.0 (28d1352db 2026-03-05)
[DIAG] wasm-pack       : wasm-pack 0.15.0
[DIAG] conda           : (not found)
[DIAG] VC++ linker     : present
[DIAG] Windows SDK     : MISSING (MSVC build needs it; or use -UseGnuToolchain)
[DIAG] install dir     : C:\Users\efebe\upspa (exists)
[DIAG] port 5432       : free
[DIAG] port 8081       : free
[DIAG] port 8082       : free
[DIAG] port 8083       : free
[DIAG] port 3000       : free
[DIAG] build mode      : GNU Rust toolchain (no MSVC / Windows SDK needed)

===== STEP 1/9 [-] 0% : Skipping install (-SkipInstall) =====

===== STEP 2/9 [##] 11% : PostgreSQL: set up the demo databases =====
setting up a self-contained PostgreSQL via conda-forge (no EnterpriseDB, no Docker)...
using conda at C:\Users\efebe\miniconda3\Scripts\conda.exe
starting PostgreSQL on port 5432...
waiting for PostgreSQL on :5432 [-] 0% (0s/60s) ... [#####] 100% (60s/60s)
  ↳ waiting for PostgreSQL on :5432 [#####] 100% (TIMEOUT after 60s)
[WARN] PostgreSQL did not become ready. Last lines of the server log:
2026-05-30 20:24:58.486 +03 [3668] ERROR: database "upspa3" already exists
2026-05-30 20:24:58.486 +03 [3668] STATEMENT: CREATE DATABASE upspa3;
2026-05-30 20:25:28.589 +03 [24352] FATAL: role "efebe" does not exist
2026-05-30 20:25:28.878 +03 [18636] FATAL: role "efebe" does not exist
2026-05-30 20:25:29.039 +03 [6056] ERROR: database "upspa1" already exists
2026-05-30 20:25:29.039 +03 [6056] STATEMENT: CREATE DATABASE upspa1;
2026-05-30 20:25:29.260 +03 [17596] ERROR: database "upspa2" already exists
2026-05-30 20:25:29.260 +03 [17596] STATEMENT: CREATE DATABASE upspa2;
2026-05-30 20:25:29.491 +03 [25900] ERROR: database "upspa3" already exists
2026-05-30 20:25:29.491 +03 [25900] STATEMENT: CREATE DATABASE upspa3;
2026-05-30 20:28:07.891 +03 [10124] LOG: checkpoint starting: time
2026-05-30 20:28:18.759 +03 [10124] LOG: checkpoint complete: wrote 99 buffers (0.6%), wrote 1 SLRU buffers; ... lsn
  ↳ =0/261B018, redo lsn=0/261AF88
2026-05-30 21:00:51.803 +03 [23856] FATAL: role "efebe" does not exist
2026-05-30 21:00:51.903 +03 [17664] FATAL: role "efebe" does not exist
2026-05-30 21:00:51.992 +03 [14600] ERROR: database "upspa1" already exists
2026-05-30 21:00:51.992 +03 [14600] STATEMENT: CREATE DATABASE upspa1;
2026-05-30 21:00:52.107 +03 [17040] ERROR: database "upspa2" already exists
2026-05-30 21:00:52.107 +03 [17040] STATEMENT: CREATE DATABASE upspa2;
2026-05-30 21:00:52.287 +03 [9468] ERROR: database "upspa3" already exists
2026-05-30 21:00:52.287 +03 [9468] STATEMENT: CREATE DATABASE upspa3;
2026-05-30 21:01:09.745 +03 [16312] LOG: background worker "logical replication launcher" (PID 26232) was terminated
  ↳ by exception OxCO00013A
2026-05-30 21:01:09.745 +03 [16312] HINT: See C include file "ntstatus.h" for a description of the hexadecimal value.
2026-05-30 21:01:09.745 +03 [16312] LOG: terminating any other active server processes
2026-05-30 21:01:09.780 +03 [16312] LOG: received fast shutdown request
~C
[WARN] pg_ctl start output:
pg_ctl.exe : pg_ctl: another server might be running; trying to start server anyway
At C:\Users\efebe\Downloads\upspa-setup-and-run.ps1:213 char:9
+ & $pgctl -D $pgData -l $pgLog -o "-p 5432" -w start 2>&1 | Ou ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (pg_ctl: another...t server anyway:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError
waiting for server to start.... done
server started
[STOP] PostgreSQL not ready. Send these files: C:\Users\efebe\upspa-demo-logs\postgres.log and C:\Users\efebe\upspa-demo-
  ↳ logs\pgctl-start.log

Done. Full log: C:\Users\efebe\upspa-demo-logs\run-20260530-210200.log
Upload that log (and any *.err.log) to Basecamp as your Step 2 feedback.

# -----
# DIAGNOSIS (added for the record):
# Port 5432 was FREE at STEP 0, so the script took the "start fresh" path. But the
# server log has NO entries after 21:01:09 (the previous server's "received fast
# shutdown request" after a Ctrl-C). If the 21:02 server had come up it would have
# logged "database system is ready to accept connections". It did not: the unclean
# shutdown left a stale postmaster.pid, so the new postgres printed "another server
# might be running" and refused to bind. pg_isready then timed out for 60s.
# The "role efebe does not exist" lines are harmless pg_isready probes defaulting to
# the OS username; they do not block readiness. The "server started" text is STALE,
# from an earlier run that used pg_ctl -w.
# FIX (script version 2026-05-30n): before starting, stop any lingering server and
# delete a stale postmaster.pid, then start and poll pg_isready on 127.0.0.1.
# -----

```

Listing 10. Windows 01-postgres.log: the PostgreSQL server log (idempotent re-creates and probe lines).

```

2026-05-30 20:24:58.486 +03 [3668] ERROR: database "upspa3" already exists
2026-05-30 20:24:58.486 +03 [3668] STATEMENT: CREATE DATABASE upspa3;
2026-05-30 20:25:28.589 +03 [24352] FATAL: role "efebe" does not exist
2026-05-30 20:25:28.878 +03 [18636] FATAL: role "efebe" does not exist

```

```

2026-05-30 20:25:29.039 +03 [6056] ERROR: database "upspa1" already exists
2026-05-30 20:25:29.039 +03 [6056] STATEMENT: CREATE DATABASE upspa1;
2026-05-30 20:25:29.260 +03 [17596] ERROR: database "upspa2" already exists
2026-05-30 20:25:29.260 +03 [17596] STATEMENT: CREATE DATABASE upspa2;
2026-05-30 20:25:29.491 +03 [25900] ERROR: database "upspa3" already exists
2026-05-30 20:25:29.491 +03 [25900] STATEMENT: CREATE DATABASE upspa3;
2026-05-30 20:28:07.891 +03 [10124] LOG: checkpoint starting: time
2026-05-30 20:28:18.759 +03 [10124] LOG: checkpoint complete: wrote 99 buffers (0.6%), wrote 1 SLRU buffers; 0 WAL file(s)
        ↳ added, 0 removed, 0 recycled; write=10.678 s, sync=0.131 s, total=10.868 s; sync files=103, longest=0.005 s,
        ↳ average=0.002 s; distance=470 kB, estimate=470 kB; lsn=0/261B018, redo lsn=0/261AF88
2026-05-30 21:00:51.803 +03 [23856] FATAL: role "efebe" does not exist
2026-05-30 21:00:51.903 +03 [17664] FATAL: role "efebe" does not exist
2026-05-30 21:00:51.992 +03 [14600] ERROR: database "upspa1" already exists
2026-05-30 21:00:51.992 +03 [14600] STATEMENT: CREATE DATABASE upspa1;
2026-05-30 21:00:52.107 +03 [17040] ERROR: database "upspa2" already exists
2026-05-30 21:00:52.107 +03 [17040] STATEMENT: CREATE DATABASE upspa2;
2026-05-30 21:00:52.287 +03 [9468] ERROR: database "upspa3" already exists
2026-05-30 21:00:52.287 +03 [9468] STATEMENT: CREATE DATABASE upspa3;
2026-05-30 21:01:09.745 +03 [16312] LOG: background worker "logical replication launcher" (PID 26232) was terminated by
        ↳ exception 0xC000013A
2026-05-30 21:01:09.745 +03 [16312] HINT: See C include file "ntstatus.h" for a description of the hexadecimal value.
2026-05-30 21:01:09.745 +03 [16312] LOG: terminating any other active server processes
2026-05-30 21:01:09.780 +03 [16312] LOG: received fast shutdown request

```

Listing 11. Windows pgctl-start.log: the PostgreSQL start output.

```

pg_ctl: another server might be running; trying to start server anyway
waiting for server to start.... done
server started

```

B Complete logs from the Linux cloud agent

The following are the raw logs from the Linux run, reproduced verbatim, including the full per-test output of the assigned package's suite.

Listing 12. Linux 00-environment-linux.log: the diagnostic snapshot and result summary.

```

UpSPA - Linux (cloud AI agent) environment
captured: 2026-05-30 20:52:54 UTC
-----
OS:      Ubuntu 24.04.4 LTS
Kernel:  Linux 6.18.5
Arch:    x86_64
CPU model: Intel(R) Xeon(R) Processor @ 2.10GHz
CPU cores: 1 vCPU
RAM:     3.9Gi total / 3.6Gi available
GPU:     none (headless CPU container; UpSPA's cryptography is CPU-only)
Node:    v22.22.2
Go:      go1.22.2
Python:  3.12.3
-----
Driver:   node upspa-demo-driver.mjs (upspa-js client compiled for Node)
Backend:  real light login server (:3000) + 3 mock SPs (:8081/2/3) each
          performing the real ristretto255 op y = k_i * blinded.
Result:   7 passed, 0 failed (see 03-driver.log)

```

Listing 13. Linux 03-driver.log: the headless driver, 7 of 7 (mock SP, real ristretto255).

```

UpSPA headless demo driver
  UID=test-user account=alice threshold=2
  login server=http://localhost:3000
  SPs: http://localhost:8081 http://localhost:8082 http://localhost:8083
  lsj=http://localhost:3000|alice

[PASS] setup / provision SPs
[PASS] register (HTTP 201)
[PASS] login (HTTP 200)
[PASS] secret update (HTTP 200)
[PASS] login after secret update (HTTP 200)
[PASS] master password update + login with new master (HTTP 200)
[PASS] resilience: login with 2 of 3 SPs (HTTP 200)

RESULT: 7 passed, 0 failed

```

Listing 14. Linux 04-crypto-test-linux.log: the full go test -v ./internal/crypto/... output: 61 test functions and two fuzz targets (63 top-level PASS entries), plus their table subtests and fuzz seeds.

```

UpSPA - internal/crypto test suite on Linux (cloud agent)
host: Ubuntu 24.04.4, Go go1.22.2, Intel Xeon, 1 vCPU, 3.9 GiB
cmd: go test -v ./internal/crypto/...
=====
=== RUN    TestCanonicalB64_RoundTrip
--- PASS: TestCanonicalB64_RoundTrip (0.00s)
=== RUN    TestCanonicalB64_AcceptsPadded
--- PASS: TestCanonicalB64_AcceptsPadded (0.00s)
=== RUN    TestCanonicalB64_RejectsInvalid
--- PASS: TestCanonicalB64_RejectsInvalid (0.00s)
=== RUN    TestDecodeFixedB64_CorrectLength
--- PASS: TestDecodeFixedB64_CorrectLength (0.00s)
=== RUN    TestDecodeFixedB64_WrongLength
--- PASS: TestDecodeFixedB64_WrongLength (0.00s)
=== RUN    TestDecodeFixedB64_InvalidBase64
--- PASS: TestDecodeFixedB64_InvalidBase64 (0.00s)
=== RUN    TestLenConstants
--- PASS: TestLenConstants (0.00s)
=== RUN    TestVerifyEd25519_ValidSig
--- PASS: TestVerifyEd25519_ValidSig (0.00s)
=== RUN    TestVerifyEd25519_InvalidSig
--- PASS: TestVerifyEd25519_InvalidSig (0.00s)
=== RUN    TestVerifyEd25519_TamperedMessage
--- PASS: TestVerifyEd25519_TamperedMessage (0.00s)
=== RUN    TestVerifyEd25519_WrongPublicKey
--- PASS: TestVerifyEd25519_WrongPublicKey (0.00s)
=== RUN    TestVerifyEd25519_PanicsOnWrongPkLength
--- PASS: TestVerifyEd25519_PanicsOnWrongPkLength (0.00s)
=== RUN    TestVerifyEd25519_PanicsOnWrongSigLength
--- PASS: TestVerifyEd25519_PanicsOnWrongSigLength (0.00s)
=== RUN    TestCanonicalB64_Negative_StandardAlphabetPlusSlash
--- PASS: TestCanonicalB64_Negative_StandardAlphabetPlusSlash (0.00s)
=== RUN    TestCanonicalB64_Negative_Punctuation
--- PASS: TestCanonicalB64_Negative_Punctuation (0.00s)
=== RUN    TestCanonicalB64_Negative_EmbeddedWhitespace
--- PASS: TestCanonicalB64_Negative_EmbeddedWhitespace (0.00s)
=== RUN    TestCanonicalB64_Negative_NullBytes
--- PASS: TestCanonicalB64_Negative_NullBytes (0.00s)
=== RUN    TestCanonicalB64_Negative_HighByteChars
--- PASS: TestCanonicalB64_Negative_HighByteChars (0.00s)
=== RUN    TestDecodeFixedB64_Negative_Ed25519PublicKey
--- PASS: TestDecodeFixedB64_Negative_Ed25519PublicKey (0.00s)
=== RUN    TestDecodeFixedB64_Negative_Ed25519Signature
--- PASS: TestDecodeFixedB64_Negative_Ed25519Signature (0.00s)
=== RUN    TestDecodeFixedB64_Negative_CtBlobNonce
--- PASS: TestDecodeFixedB64_Negative_CtBlobNonce (0.00s)
=== RUN    TestDecodeFixedB64_Negative_CtBlobTag
--- PASS: TestDecodeFixedB64_Negative_CtBlobTag (0.00s)
=== RUN    TestDecodeFixedB64_Negative_Ristretto
--- PASS: TestDecodeFixedB64_Negative_Ristretto (0.00s)
=== RUN    TestDecodeFixedB64_Negative_ScalarKi
--- PASS: TestDecodeFixedB64_Negative_ScalarKi (0.00s)
=== RUN    TestDecodeFixedB64_Negative_InvalidBase64WinsOverWrongLength
--- PASS: TestDecodeFixedB64_Negative_InvalidBase64WinsOverWrongLength (0.00s)
=== RUN    TestRistrettoScalarMult_Negative_InvalidPoints
--- PASS: TestRistrettoScalarMult_Negative_InvalidPoints/high-bit_set_(0xFF*32) (0.00s)
=== RUN    TestRistrettoScalarMult_Negative_InvalidPoints/last_byte_0x80
--- PASS: TestRistrettoScalarMult_Negative_InvalidPoints/non-canonical_field_element_(p+1) (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidPoints (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidPoints/high-bit_set_(0xFF*32) (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidPoints/last_byte_0x80 (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidPoints/non-canonical_field_element_(p+1) (0.00s)
=== RUN    TestRistrettoScalarMult_Negative_InvalidScalars
--- PASS: TestRistrettoScalarMult_Negative_InvalidScalars/all_0xFF_(>>_1) (0.00s)
=== RUN    TestRistrettoScalarMult_Negative_InvalidScalars/all_0x7F_(>_1)
--- PASS: TestRistrettoScalarMult_Negative_InvalidScalars/scalar_equals_1 (0.00s)
=== RUN    TestRistrettoScalarMult_Negative_InvalidScalars/scalar_equals_l+1
--- PASS: TestRistrettoScalarMult_Negative_InvalidScalars (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidScalars/all_0xFF_(>>_1) (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidScalars/all_0x7F_(>_1) (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidScalars/scalar_equals_1 (0.00s)
--- PASS: TestRistrettoScalarMult_Negative_InvalidScalars/scalar_equals_l+1 (0.00s)
=== RUN    TestRistrettoScalarMult_Negative_ScalarWrongLengths
--- PASS: TestRistrettoScalarMult_Negative_ScalarWrongLengths (0.00s)
=== RUN    TestRistrettoScalarMult_Negative_PointWrongLengths
--- PASS: TestRistrettoScalarMult_Negative_PointWrongLengths (0.00s)
=== RUN    TestVerifyEd25519_Negative_ShortKeyPanics
--- PASS: TestVerifyEd25519_Negative_ShortKeyPanics (0.00s)
=== RUN    TestVerifyEd25519_Negative_LongKeyPanics
--- PASS: TestVerifyEd25519_Negative_LongKeyPanics (0.00s)
=== RUN    TestVerifyEd25519_Negative_ShortSigPanics
--- PASS: TestVerifyEd25519_Negative_ShortSigPanics (0.00s)
=== RUN    TestVerifyEd25519_Negative_LongSigPanics
--- PASS: TestVerifyEd25519_Negative_LongSigPanics (0.00s)

```

```

=== RUN    TestBuildPwdUpdateSigMsg_TotalLength_EmptyCt
--- PASS: TestBuildPwdUpdateSigMsg_TotalLength_EmptyCt (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_TotalLength_NonEmptyCt
--- PASS: TestBuildPwdUpdateSigMsg_TotalLength_NonEmptyCt (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_Nonce_Offset
--- PASS: TestBuildPwdUpdateSigMsg_Nonce_Offset (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_Ct_Offset
--- PASS: TestBuildPwdUpdateSigMsg_Ct_Offset (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_Ct_Empty
--- PASS: TestBuildPwdUpdateSigMsg_Ct_Empty (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_Tag_Offset
--- PASS: TestBuildPwdUpdateSigMsg_Tag_Offset (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_KiNew_Offset
--- PASS: TestBuildPwdUpdateSigMsg_KiNew_Offset (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_Timestamp_LittleEndian
--- PASS: TestBuildPwdUpdateSigMsg_Timestamp_LittleEndian (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_Timestamp_KnownBytes
--- PASS: TestBuildPwdUpdateSigMsg_Timestamp_KnownBytes (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_Timestamp_Zero
--- PASS: TestBuildPwdUpdateSigMsg_Timestamp_Zero (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_SpID_LittleEndian
--- PASS: TestBuildPwdUpdateSigMsg_SpID_LittleEndian (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_SpID_KnownBytes
--- PASS: TestBuildPwdUpdateSigMsg_SpID_KnownBytes (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_SpID_Zero
--- PASS: TestBuildPwdUpdateSigMsg_SpID_Zero (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_GoldenVector
--- PASS: TestBuildPwdUpdateSigMsg_GoldenVector (0.00s)
=== RUN    TestBuildPwdUpdateSigMsg_NoAlias
--- PASS: TestBuildPwdUpdateSigMsg_NoAlias (0.00s)
=== RUN    TestRistrettoScalarMult_ValidInputs
--- PASS: TestRistrettoScalarMult_ValidInputs (0.00s)
=== RUN    TestRistrettoScalarMult_Deterministic
--- PASS: TestRistrettoScalarMult_Deterministic (0.00s)
=== RUN    TestRistrettoScalarMult_DifferentKeys_DifferentOutputs
--- PASS: TestRistrettoScalarMult_DifferentKeys_DifferentOutputs (0.00s)
=== RUN    TestRistrettoScalarMult_OneTimesG_RoundTrip
--- PASS: TestRistrettoScalarMult_OneTimesG_RoundTrip (0.00s)
=== RUN    TestRistrettoScalarMult_InvalidPoint
--- PASS: TestRistrettoScalarMult_InvalidPoint (0.00s)
=== RUN    TestRistrettoScalarMult_InvalidScalar
--- PASS: TestRistrettoScalarMult_InvalidScalar (0.00s)
=== RUN    TestRistrettoScalarMult_WrongScalarLength
--- PASS: TestRistrettoScalarMult_WrongScalarLength (0.00s)
=== RUN    TestRistrettoScalarMult_WrongPointLength
--- PASS: TestRistrettoScalarMult_WrongPointLength (0.00s)
=== RUN    TestRistrettoScalarMult_IdentityPoint_IsValid
--- PASS: TestRistrettoScalarMult_IdentityPoint_IsValid (0.00s)
=== RUN    TestGenerateScalarKi_Length
--- PASS: TestGenerateScalarKi_Length (0.00s)
=== RUN    TestGenerateScalarKi_IsCanonical
--- PASS: TestGenerateScalarKi_IsCanonical (0.01s)
=== RUN    TestGenerateScalarKi_Unique
--- PASS: TestGenerateScalarKi_Unique (0.00s)
=== RUN    TestGenerateScalarKi_NotAllZeros
--- PASS: TestGenerateScalarKi_NotAllZeros (0.00s)
=== RUN    FuzzCanonicalB64
=== RUN    FuzzCanonicalB64/seed#0
=== RUN    FuzzCanonicalB64/seed#1
=== RUN    FuzzCanonicalB64/seed#2
=== RUN    FuzzCanonicalB64/seed#3
=== RUN    FuzzCanonicalB64/seed#4
=== RUN    FuzzCanonicalB64/seed#5
=== RUN    FuzzCanonicalB64/seed#6
=== RUN    FuzzCanonicalB64/seed#7
=== RUN    FuzzCanonicalB64/seed#8
=== RUN    FuzzCanonicalB64/seed#9
=== RUN    FuzzCanonicalB64/seed#10
=== RUN    FuzzCanonicalB64/seed#11
=== RUN    FuzzCanonicalB64/seed#12
=== RUN    FuzzCanonicalB64/seed#13
=== RUN    FuzzCanonicalB64/seed#14
=== RUN    FuzzCanonicalB64/seed#15
=== RUN    FuzzCanonicalB64/seed#16
--- PASS: FuzzCanonicalB64 (0.00s)
--- PASS: FuzzCanonicalB64/seed#0 (0.00s)
--- PASS: FuzzCanonicalB64/seed#1 (0.00s)
--- PASS: FuzzCanonicalB64/seed#2 (0.00s)
--- PASS: FuzzCanonicalB64/seed#3 (0.00s)
--- PASS: FuzzCanonicalB64/seed#4 (0.00s)
--- PASS: FuzzCanonicalB64/seed#5 (0.00s)
--- PASS: FuzzCanonicalB64/seed#6 (0.00s)
--- PASS: FuzzCanonicalB64/seed#7 (0.00s)
--- PASS: FuzzCanonicalB64/seed#8 (0.00s)

```

```

--- PASS: FuzzCanonicalB64/seed#9 (0.00s)
--- PASS: FuzzCanonicalB64/seed#10 (0.00s)
--- PASS: FuzzCanonicalB64/seed#11 (0.00s)
--- PASS: FuzzCanonicalB64/seed#12 (0.00s)
--- PASS: FuzzCanonicalB64/seed#13 (0.00s)
--- PASS: FuzzCanonicalB64/seed#14 (0.00s)
--- PASS: FuzzCanonicalB64/seed#15 (0.00s)
--- PASS: FuzzCanonicalB64/seed#16 (0.00s)
=== RUN    FuzzDecodeFixedB64
=== RUN    FuzzDecodeFixedB64/seed#0
=== RUN    FuzzDecodeFixedB64/seed#1
=== RUN    FuzzDecodeFixedB64/seed#2
=== RUN    FuzzDecodeFixedB64/seed#3
=== RUN    FuzzDecodeFixedB64/seed#4
=== RUN    FuzzDecodeFixedB64/seed#5
=== RUN    FuzzDecodeFixedB64/seed#6
=== RUN    FuzzDecodeFixedB64/seed#7
--- PASS: FuzzDecodeFixedB64 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#0 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#1 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#2 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#3 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#4 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#5 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#6 (0.00s)
--- PASS: FuzzDecodeFixedB64/seed#7 (0.00s)
PASS
ok      upspa/internal/crypto 0.013s

```

Listing 15. Linux 05-go-sp-build-linux.log: the build record, including why the full-service link is exercised on Windows.

```

UpSPA - Go Storage Provider build on Linux (cloud agent)
host: Ubuntu 24.04.4, Go 1.22.2, Intel Xeon, 1 vCPU, 3.9 GiB
=====

internal/crypto (Efe's assigned package)
cmd:    go test ./internal/crypto/...
result: ok upspa/internal/crypto (63 PASS, 0 FAIL; see 04-crypto-test-linux.log)
deps:   github.com/gtank/ristretto255 only -> compiles cleanly on Linux.

Full service (go build ./...)
The full binary additionally links jackc/pgx (PostgreSQL driver), which depends on
golang.org/x/{sync, text, crypto}. The sandbox network does not reach the Go module
proxy / golang.org; routing those through the github.com/golang mirror instead pins
module versions whose go.mod toolchain directive exceeds this sandbox's Go 1.22.2.
The full-service link was therefore not exercised on Linux. This is a sandbox
network/toolchain limitation, not a code issue.

Cross-environment summary for the Go component
Windows host (Go 1.26): full Storage Provider builds and runs (Postgres-backed),
exercised end-to-end via the setup script.
Linux (Go 1.22.2): the assigned internal/crypto package builds and its full
test suite passes (63/63).

```

Listing 16. Linux indicative timing: commands and raw output (medians; mock SP, localhost).

```

UpSPA indicative timing on Linux (cloud agent)
host: Ubuntu 24.04.4, Node v22.22.2, 1 vCPU, 3.9 GiB; mock SP (real ristretto255 via @noble) over localhost
method: process.hrtime.bigint() deltas in the Node driver; medians (60 trials for the TOPRF round, 40 for auth/login)

$ NSP=3 T=2 node timing.mjs
{"config":"2-of-3","toprf_round_ms":19.88,"auth_crypto_ms":18.03,"login_e2e_ms":59.8}
$ NSP=5 T=3 node timing.mjs
{"config":"3-of-5","toprf_round_ms":23.04,"auth_crypto_ms":26.16,"login_e2e_ms":70.16}
$ NSP=7 T=4 node timing.mjs
{"config":"4-of-7","toprf_round_ms":28.35,"auth_crypto_ms":30.75,"login_e2e_ms":81.95}
$ node timing1.mjs # one-shot, 2-of-3
{"register_ms":176.92,"secret_update_ms":114.89,"master_pw_update_ms":42.3}

```

C Issues surfaced during deployment, in full

These issues were found while exercising the full stack repeatedly. Each is a robustness or developer-experience fix; none affects the cryptography or the clean-database 7 of 7 result.

Stale lock stalls PostgreSQL start.

On repeated runs the background PostgreSQL start (launched hidden) failed to become ready within the 60-second window, and no *new* server log was written by that start (the file holds the prior server's entries, ending at its clean

shutdown), meaning that start never reached the point of opening its log. The recorded diagnosis (Appendix A) is a stale `postmaster.pid`: a prior server had not exited gracefully (a Ctrl-C is a clean fast shutdown and would not leave the lock), so the next start printed “another server might be running” and aborted before binding. Starting the server in the foreground with the same data directory confirmed the cluster itself is healthy and comes up. The fix is to clear the stale `postmaster.pid` and poll `pg_isready` before proceeding; the script’s self-heal does this (a one-off foreground start also clears it).

Headless-driver non-idempotency.

The driver uses a fixed identity (`alice / test-password`) and rotates the master password to `new-password` mid-run, leaving that persisted; a second run against non-empty databases then fails registration (`alice` already exists) and login (the password was rotated): only the provisioning step passes and the dependent phases fail. A rejected login also returns HTTP 200 with the HTML login form rather than a 4xx, so the failure surfaces as raw HTML. Dropping and recreating the three databases restores 7 of 7. Fixes: reset the demo databases at start, give the driver a unique account per run, or have the login server return a non-200 on failed login.

Native extension build on `options.html`.

The `@crxjs 1.0.14` path bug embeds an absolute Windows path whose backslashes parse as invalid string escapes; the applied build fixes cover `background/index.ts` and the Vite alias but not the HTML entry points, so no `dist/` is produced and the prebuilt dist is used instead. Fixes: upgrade `@crxjs` past 1.0.14, or force forward-slash module IDs globally in the Vite/Rollup config.

Packaging note.

The prebuilt archive extracts with its `upspa-extension-dist/` directory nested twice, so `-UsePrebuiltDist` must point at the inner folder.

Area	Result
Protocol (headless, clean database)	7 of 7 pass
WASM build (GNU toolchain)	OK
Storage Providers + login server	OK
PostgreSQL 18.4 (after foreground recovery)	OK
Native extension build	fails (<code>@crxjs 1.0.14, options.html</code>)
Prebuilt dist fallback	OK